Contents

Monitoring & Observability — Deep Dive Notes **1**
1. The Four Golden Signals (Google SRE) . . . 1
2. Observability Pillars . . . 1
   - Metrics . . . 1
   - Logs . . . 2
   - Traces . . . 2
   - OpenTelemetry . . . 2
3. Prometheus Architecture . . . 2
   - Metric Types . . . 2
4. PromQL — Key Patterns . . . 2
5. Alerting Rules Best Practices . . . 3
   - Alert Design Principles . . . 3
6. SLO-Based Alerting . . . 4
   - Concepts . . . 4
   - Burn Rate Alerting (Multi-Window) . . . 4
   - Error Budget Dashboard Components . . . 4
7. Logging Architecture . . . 4
   - Structured Logging (JSON) . . . 4
   - Log Levels . . . 5
   - Fluent Bit on Kubernetes . . . 5
   - Loki + Grafana (Lightweight ELK Alternative) . . . 5
8. Distributed Tracing . . . 5
   - Trace Concepts . . . 5
   - Trace Propagation Headers . . . 5
   - Sampling Strategies . . . 6
9. Grafana Dashboard Design . . . 6
   - Essential Panels per Service . . . 6
   - Variables for Dynamic Dashboards . . . 6
10. On-Call Best Practices . . . 6
    - Incident Severity Levels . . . 6
    - Incident Response Flow . . . 7
- Resources . . . 7

Monitoring & Observability — Deep Dive Notes

1. The Four Golden Signals (Google SRE)

Signal	Definition	Example Metric
Latency	Time to serve a request	p99 response time < 200ms
Traffic	Demand on the system	Requests per second
Errors	Rate of failed requests	HTTP 5xx / total * 100
Saturation	How “full” the service is	CPU %, queue depth, connection pool %

USE Method (for resources): Utilization + Saturation + Errors **RED Method** (for services): Rate + Errors + Duration

---

2. Observability Pillars

Metrics

- Aggregated numeric data over time
- Low storage cost, fast queries
- Tools: Prometheus, CloudWatch, Datadog, VictoriaMetrics

Logs

- Discrete events with full context
- High storage cost, rich information
- Tools: ELK Stack, Loki+Grafana, Splunk, CloudWatch Logs

Traces

- Request journey across distributed services
- Shows latency per hop, identifies bottlenecks
- Tools: Jaeger, Tempo, Zipkin, AWS X-Ray, Datadog APM

OpenTelemetry

- CNCF standard for unified instrumentation
- Single SDK → export to any backend
- Auto-instrumentation for popular frameworks
- Reduces vendor lock-in

---

3. Prometheus Architecture

```
Applications/Exporters  →  Prometheus Server  →  Grafana
   (expose /metrics)        (scrape + store)     (visualize)
                                  ↓
                           Alertmanager  →  PagerDuty / Slack
```

Metric Types

Counter — only goes up; reset on restart

```
http_requests_total{method="GET", status="200"} 12345
```

Gauge — can go up or down

```
node_memory_available_bytes 2147483648
kube_deployment_spec_replicas{deployment="api"} 3
```

Histogram — samples + configurable buckets; use for latency

```
http_request_duration_seconds_bucket{le="0.1"} 9800
http_request_duration_seconds_bucket{le="0.5"} 11234
http_request_duration_seconds_sum 5432.1
http_request_duration_seconds_count 12000
```

Summary — client-side pre-calculated quantiles (less flexible)

---

4. PromQL — Key Patterns

```
# Requests per second over last 5 minutes
rate(http_requests_total[5m])

# Error rate as percentage
rate(http_requests_total{status=~"5.."}[5m])
  / rate(http_requests_total[5m]) * 100

# 99th percentile latency from histogram
histogram_quantile(0.99,
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le, service)
)
```

```
# CPU throttling ratio
rate(container_cpu_cfs_throttled_seconds_total[5m])
  / rate(container_cpu_cfs_periods_total[5m]) * 100

# Memory usage vs limit
container_memory_working_set_bytes
  / container_spec_memory_limit_bytes * 100

# Pods not ready
kube_deployment_spec_replicas
  - kube_deployment_status_ready_replicas

# Top 5 pods by memory
topk(5, container_memory_working_set_bytes{container!=""})

# Alert: sustained high error rate
rate(http_requests_total{code=~"5.."}[5m]) > 0.05
  and on() rate(http_requests_total[5m]) > 10

# Increase in value over 1 hour
increase(errors_total[1h])

# Predict disk fill (4h ahead)
predict_linear(node_filesystem_free_bytes[2h], 4 * 3600) < 0
```

---

5. Alerting Rules Best Practices

```yaml
# Good alert anatomy
- alert: HighErrorRate
  # Expression: must be unambiguous, no false positives
  expr: |
    sum(rate(http_requests_total{status=~"5.."}[5m])) by (service)
    /
    sum(rate(http_requests_total[5m])) by (service)
    * 100 > 5
  # for: ensure it's sustained, not a blip
  for: 2m
  labels:
    severity: critical   # critical / warning / info
    team: platform
  annotations:
    summary: "High error rate on {{ $labels.service }}"
    description: "Error rate {{ printf \"%.1f\" $value }}% for last 2m"
    runbook_url: "https://runbooks.company.com/high-error-rate"
    dashboard_url: "https://grafana.company.com/d/xyz"
```

Alert Design Principles

1. **Alert on symptoms, not causes** — “users can’t check out” not “DB CPU high”
2. **Every alert needs a runbook** — on-call should know exactly what to do
3. **Alert on burn rate, not thresholds** — SLO-based alerts reduce noise
4. **Avoid alert fatigue** — if ignored, it’s useless noise; remove or fix
5. **Test alerts** — use `amtool` to test routes, fire test alerts in staging

---

6. SLO-Based Alerting

Concepts

```
SLI (Service Level Indicator) — what you measure
  "ratio of successful requests to total requests"

SLO (Service Level Objective) — your target
  "99.9% of requests complete successfully over 30 days"

Error Budget = 1 - SLO = 0.1% = 43.8 min/month
  When budget is gone → freeze deployments, fix reliability
```

Burn Rate Alerting (Multi-Window)

```
Burn Rate = actual error rate / allowed error rate

If SLO = 99.9%:
  Allowed error rate = 0.001 (0.1%)
  If actual = 1%, burn rate = 10x
  At 10x burn rate, budget exhausted in 3 days

Alert thresholds:
  Critical (page): 14.4x burn rate sustained for 1h
    → exhausts 2% of monthly budget in 1 hour
  Warning (ticket): 6x burn rate sustained for 6h
    → exhausts 5% of monthly budget in 6 hours
  Info: 1x burn rate sustained for 3 days
    → on track to exhaust full budget
```

Error Budget Dashboard Components

- Remaining error budget % (gauge)
- Budget burn rate over time (graph)
- Recent deployments overlaid (annotations)
- Projected budget exhaustion date

---

7. Logging Architecture

Structured Logging (JSON)

```json
{
  "timestamp": "2026-05-25T14:32:01.123Z",
  "level": "ERROR",
  "service": "payment-api",
  "trace_id": "abc123def456",
  "span_id": "789xyz",
  "user_id": "u_12345",
  "order_id": "ord_67890",
  "message": "Payment processing failed",
  "error": "connection timeout after 5000ms",
  "duration_ms": 5001,
  "http.method": "POST",
  "http.path": "/v2/payments",
  "http.status": 503
}
```

Log Levels

Level	Use
TRACE	Extremely detailed (disable in production)
DEBUG	Diagnostic info (disable in production)
INFO	Normal operation events
WARN	Unexpected but handled situation
ERROR	Operation failed, needs investigation
FATAL	System cannot continue, immediate attention

Fluent Bit on Kubernetes

```
[INPUT]  tail /var/log/containers/*.log
[FILTER] kubernetes metadata enrichment (pod, namespace, labels)
[FILTER] parser (JSON, regex)
[FILTER] record_modifier (add cluster name, environment)
[OUTPUT] opensearch / loki / cloudwatch
```

Loki + Grafana (Lightweight ELK Alternative)

- Loki: index only labels, not full text (much cheaper than Elasticsearch)
- Promtail: log shipper (like Filebeat but for Loki)
- LogQL query language (similar to PromQL):

```
# Errors from production API
{namespace="production", app="api"} |= "ERROR"

# Rate of errors per minute
rate({namespace="production"} |= "ERROR" [1m])

# Parse JSON and filter
{app="api"} | json | status >= 500

# Latency from structured logs
{app="api"} | json | duration_ms > 1000

# Count by status code
sum by (status) (
  rate({app="api"} | json | unwrap status [5m])
)
```

---

8. Distributed Tracing

Trace Concepts

```
Trace:  A complete request flow (has unique trace_id)
Span:   A single operation within a trace (has span_id, parent_span_id)
Tags:   Key-value metadata on a span
Logs:   Events within a span (errors, milestones)
Baggage: Propagated data across services
```

Trace Propagation Headers

```
W3C Trace Context (standard):
  traceparent: 00-traceid-spanid-flags
  tracestate: vendor-specific state
```

```
B3 (Zipkin/older):
  X-B3-TraceId, X-B3-SpanId, X-B3-ParentSpanId, X-B3-Sampled
```

Sampling Strategies

- **Head-based (random)**: 1-10% sample at trace start. Simple, misses rare errors.
- **Tail-based**: Buffer entire trace, sample based on outcome (all errors, slow traces). Expensive but precise.
- **OpenTelemetry Collector**: enables tail sampling without app changes.

---

9. Grafana Dashboard Design

Essential Panels per Service

```
Row 1: Traffic
  - Requests/sec (rate graph)
  - Active connections (gauge)

Row 2: Errors
  - Error rate % (stat + threshold coloring)
  - Error count by type (bar chart)

Row 3: Latency
  - p50 / p95 / p99 heatmap or time series
  - Latency distribution (heatmap)

Row 4: Saturation
  - CPU usage vs limit (graph)
  - Memory usage vs limit (graph)
  - Pod restart count (table)

Row 5: Dependencies
  - Database query time
  - Cache hit rate
  - External API error rate
```

Variables for Dynamic Dashboards

```
Namespace: query → label_values(kube_pod_info, namespace)
Deployment: query → label_values(kube_deployment_info{namespace=~"$namespace"}, deployment)
Interval: interval → 1m, 5m, 15m, 1h
```

---

10. On-Call Best Practices

Incident Severity Levels

Level	Definition	Response Time
SEV-1	Complete outage, all users affected	Immediate
SEV-2	Major feature broken, most users affected	< 15 min
SEV-3	Degraded performance, some users affected	< 1 hour
SEV-4	Minor issue, few users, workaround exists	Next business day

Incident Response Flow

```
1. Detect    — alert fires (automated) or user report
2. Triage    — assess severity, assign IC (Incident Commander)
3. Declare   — create incident channel (#inc-YYYYMMDD-title)
4. Mitigate  — restore service (rollback, scale, redirect)
5. Resolve   — confirm recovery, all metrics green
6. Follow-up — blameless postmortem within 48-72 hours
```

Resources

- https://prometheus.io/docs/ — Prometheus official docs
- https://grafana.com/docs/ — Grafana documentation
- https://opentelemetry.io/docs/ — OpenTelemetry
- https://sre.google/workbook/alerting-on-slos/ — Google SRE alerting
- https://www.robustperception.io/blog — Prometheus deep dives by Julius Volz
- https://promlabs.com/promql-cheat-sheet/ — PromQL reference
- https://github.com/grafana/loki — Loki docs